

John (Jack) Leniart

CSC 555 – Final Project

Introduction to the Dataset

The dataset I chose to use for this project was the Fraudulent Transactions Prediction dataset from Kaggle. The dataset contains simulated financial transactions. Some of the transactions are labeled as legitimate and some are labeled as fraudulent.

Each transaction is represented by a row in the dataset. There are columns containing data for the time of the transaction, information about the customer (account owner), and the requested amount. There are also columns that contain the account balances before and after the transaction was run.

The dataset contains 6,362,620 rows and 11 columns. The csv file containing the dataset is 470 MB. The author did not explicitly state how this dataset was created; however, based on discussion from other users on the dataset page in Kaggle, it appears that this is simulated data that was created using a website called Paysim.

Kaggle Dataset: <https://www.kaggle.com/datasets/vardhansiramdasu/fraudulent-transactions-prediction/data>

After familiarizing myself with the dataset, my initial thought was to develop a classification model to predict whether or not a transaction is fraudulent. The model could then be used by banks and other financial institutions to help prevent fraud and notify their customers as soon as potentially fraudulent activity takes place.

Before a classification model could be developed, I needed to perform some data preprocessing on the original dataset as well as Exploratory Data Analysis.

Preprocessing

The type variable contains string values indicating the type of transaction that the row is representing. There are five possible values: cash in, cash out, debit, payment, and transfer. I decided to create dummy variables to represent these five types with numeric binary values.

There were also some variables that I thought could be dropped from the dataset. I dropped the step variable, which contains an integer that indicates which hour during the 30-day simulation the transaction occurred. My reasoning for dropping this variable is that it would just add noise to the model. If this variable had represented the actual time of day, I think it would have been worth keeping.

I also dropped the nameOrig and nameDest variables. Those two variables contain a string of letters and numbers indicating the originating and destination accounts. Since most of the values for these variables are unique strings, I do not think they will provide any insight to our model.

Lastly, I created two new variables: BalChangeOrig and BalChangeDest. These variables will store the change in the balance of the originating account and the destination account for each transaction.

Exploratory Data Analysis

To start my analysis, I looked at the value counts for the categorical variables. I started with the dependent variable, isFraud. There are 6,354,407 transactions labeled as legitimate (isFraud = 0) and 8,213 transactions labeled as fraudulent (isFraud = 1).

This imbalance can make it difficult to develop a smart model that can accurately predict fraud. Most models would lean heavily towards not predicting fraud because the dataset they are being trained on shows there is a much higher probability that transactions are legitimate.

I also looked at the counts for the variable type which has values: cash in, cash out, debit, payment, and transfer. The cash out and payment types appear the most in the dataset. Followed by cash in, transfer, and debit. The transfer and debit types are noticeably underrepresented in the dataset.

		+-----+-----+		
		type count		
		+-----+-----+		
+-----+-----+	isFraud count	CASH_OUT 2237500		
		CASH_IN 1399284		
		PAYMENT 2151495		
		DEBIT 41432		
		TRANSFER 532909		
		+-----+-----+		
		type count		
		+-----+-----+		
+-----+-----+	1 8213			
		+-----+-----+		
		type count		
		+-----+-----+		
+-----+-----+	0 6354407			
		+-----+-----+		

One area I would focus on for future development of these classification models is using stratified sampling when possible or creating synthetic data to make sure there are not any major imbalances between the variable classes in the dataset.

I also looked at some descriptive statistics for the numeric variables to get an understanding of the distribution of their values. The first thing that jumps out is the large range of values and high standard deviation for all of the variables. There are many rows in the dataset where the values for the originating account or the destination account are all zero. This coupled with the larger balances and amounts really stretches out the distributions of these variables.

		+-----+-----+		+-----+-----+		+-----+-----+		+-----+-----+	
summary	amount	oldbalanceOrig	newbalanceOrig	BalChangeOrig	oldbalanceDest	newbalanceDest	BalChangeDest		
		+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+
count	6362620	6362620	6362620	6362620	6362620	6362620	6362620	6362620	6362620
mean	179861.90355834848	833883.104048853	855113.668553137	21230.564503113146	1100701.6665187487	1224996.3981876222	124294.73167288191		
stddev	603858.2316747018	2888242.6729981783	2924048.502909438	146643.2864296386	3399180.1117589492	3674128.94101535	812939.0805457995		
min	0.0	0.0	0.0	-1.0000002E7	0.0	0.0	-1.3060826E7		
max	9.244552E7	5.958504E7	4.958504E7	1915268.0	3.56015904E8	3.56179264E8	1.0568784E8		
		+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+	+-----+-----+

Classification Models

For this project, I developed three different classification models: Linear Support Vector Machine, Random Forest, and Decision Tree. All models were developed using 5-fold cross-validation.

I started with Linear SVM, which felt like a good fit since our dependent variable only has two classes (binary). The SVM model turned out to be very biased towards predicting no fraud. The confusion matrix showed that the model predicted isFraud=0 for all rows in the test dataset. As I mentioned earlier, this was something I was

concerned about. Despite that, the model still had a high weighted accuracy. This is because the vast majority of the rows in the test dataset were not fraud.

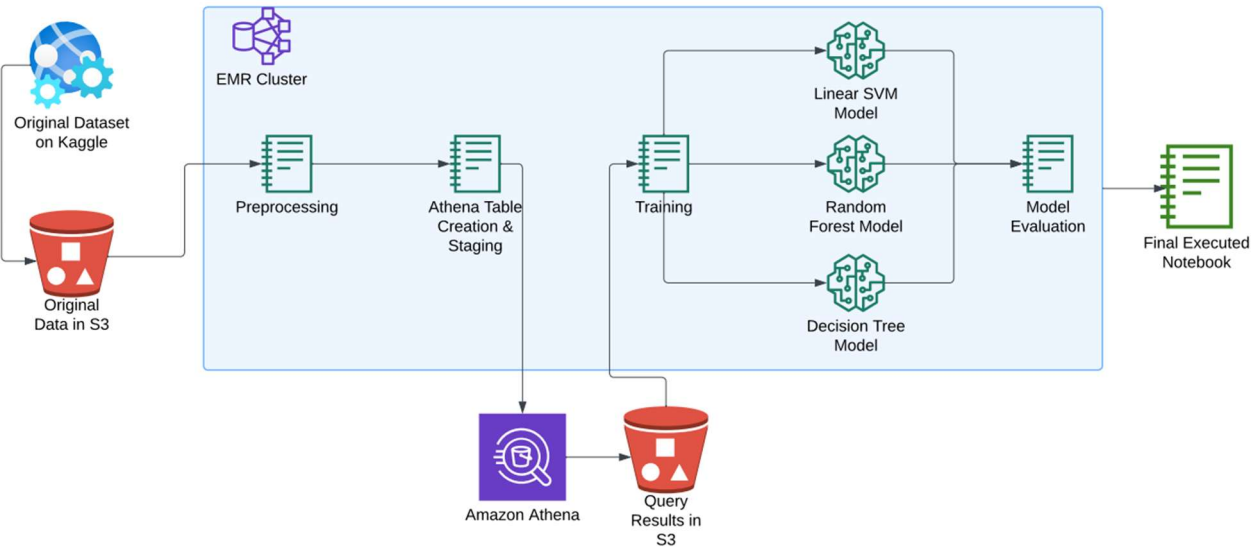
The second model I developed was a Random Forest model. The model had an unweighted accuracy of 84.76%, which was a huge increase compared to SVM, and all of the weighted metrics improved as well.

The third and final model I developed was a Decision Tree model. I expected this to have very similar metrics to the Random Forest model as random Forest is just an ensemble of Decision Trees, and my intuition was correct. The weighted metrics for the Decision Tree model were nearly identical to those of the Random Forest model. Surprisingly, the unweighted accuracy improved to 87.92%.

Model	UnweightedAccuracy	WeightedAccuracy	WeightedPrecision	WeightedRecall	WeightedF1
Linear SVM	0.5	0.9985	0.997	0.9985	0.9977
Random Forest	0.8476	0.9995	0.9995	0.9995	0.9995
Decision Tree	0.8792	0.9995	0.9995	0.9995	0.9995

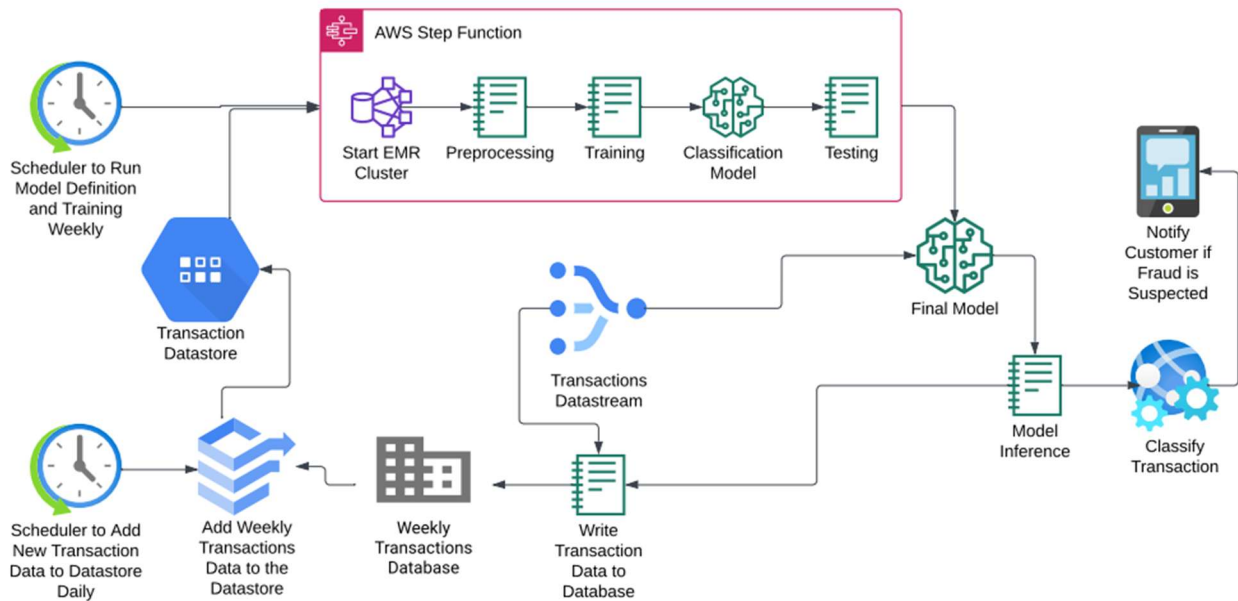
Architecture Diagrams

Now, let’s look an architecture diagram of the steps and processes I used for this project. We start with the original data being sourced from the web (Kaggle) and stored in S3. After starting up an EMR Cluster and notebook in AWS, we read the original data from S3 into a Spark dataframe for preprocessing. The preprocessed data is then staged in Athena, where we can query it efficiently. Athena query results are written to S3, and those results are then read back into a Spark dataframe within the notebook. Next, we split the data into training and testing sets and develop our three classification models. The models are trained, tested, and evaluated within the notebook.



Next, let's look at a diagram that shows how this process could be improved and expanded in the future. The main steps I completed in my final project are now housed in an AWS step function. I added a Scheduler job to execute this step function once a week and update the classification model using the latest data.

I also added a Datastream for all incoming new data and a Datastore to hold all historical data. The Datastream writes all transactions to a database, and then a Scheduler job runs once a week to add all of the week's transactions to the main Datastore. All new transactions that are received through the Datastream are run through the latest model to predict whether or not the transaction is fraudulent. The transaction is classified and the customer is notified if fraud is suspected.



Challenges Faced and Conclusion

There were many challenges that I faced during the course of this project. Most due to the fact that I am new to AWS and PySpark. I spent many hours searching for sample code and then going through the process of trial and error. Despite the challenges, it was a great learning opportunity for me.

I have limited experience working with very large datasets, so it was also a challenge to get the data in place and then work with it. The amount of time spent waiting for code to run was frustrating at times. However, this inconvenience highlights the value of distributed computing. Without the use of distributed computing on the AWS platform, I would not have been able to analyze this dataset.

The Random Forest and Decision Tree models performed the best when predicting the dependent variable for our dataset. For future development of this project, those would be the two model types that I would focus on.

As I mentioned earlier, models like this could be useful to banks and financial institutions to safeguard against fraud. Although, the larger institutions already have much more complex and accurate models and algorithms that they use on a daily basis.

For myself, I would like to develop this model further and specifically focus on correcting the class imbalance for the dependent variable. I would like to try stratified sampling and oversampling techniques to see if that could help the model perform even better.